Tarefa 18: Comparando Soma de Vetores entre CPU e GPU em OpenMP

Matheus Bezeril Pinto

Outubro de 2026

Resumo

Este relatório apresenta a implementação e a análise das execuções de adição de vetores em OpenMP, seguindo os exercícios de CPU e GPU indicados na tarefa 18. O programa soma dez milhões de elementos em precisão simples, verifica o resultado e mede o tempo da chamada de soma. Foram analisadas sete séries de cinco medições, além de quatro execuções identificadas como aquecimento. Na CPU, os tempos médios foram de 10,568 ms e 9,975 ms com uma thread configurada, e 5,379 ms com quatro. Os logs do modo GPU registraram médias entre 5,997 ms e 13,484 ms em nós que listam aceleradores V100 ou H200. Todos os registros apresentaram zero erros. Contudo, os arquivos não comprovam a execução efetiva do cálculo no acelerador: há limitações na compilação, na confirmação do dispositivo e no tratamento dos vetores alocados dinamicamente. O relatório explica essas diferenças, compara os tempos observados e apresenta os ajustes necessários para concluir uma comparação CPU–GPU verificada. Os códigos, os registros completos e a inspeção dos binários estão nos apêndices.

1 Objetivo e material analisado

O enunciado solicita realizar os exercícios de soma de vetores do tutorial de OpenMP para GPUs em um nó com GPU do NPAD, comparar a execução somente na CPU com o uso da GPU e relatar problemas e soluções [1, 2]. O objetivo inclui compreender onde o cálculo executa e quais custos entram na medição.

Foram analisados `t18.c`, `somas.c`, `aux18.c`, `jobs.sh`, os sete arquivos de saída e os dois executáveis enviados ao GitHub no commit `e17b7d0`, presentes na revisão `9118003` [3]. Os executáveis foram inspecionados estaticamente, sem produzir novas medições. O relatório preserva os registros disponíveis e distingue as melhorias já presentes no código dos ajustes ainda propostos.

O enunciado identifica os exercícios pelos slides 27 e 48. Na edição SC23 do tutorial disponível no repositório NPAD consultado, os exercícios equivalentes de soma na CPU e na GPU aparecem nas páginas 29 e 50 do PDF. Essa diferença de paginação não altera a atividade: o primeiro usa distribuição do laço na CPU; o segundo introduz `target` e `loop` para offload.

2 O problema: adição de vetores

2.1 Operação e independência das iterações

Dados dois vetores a e b de tamanho N, calcula-se

$$c_i = a_i + b_i, \qquad i = 0, \ldots, N-1.$$

O elemento c_i depende apenas de a_i e b_i. Não há uma soma acumulada entre os índices, e uma iteração não precisa do resultado da anterior. Essa independência permite distribuir as iterações entre unidades de execução.

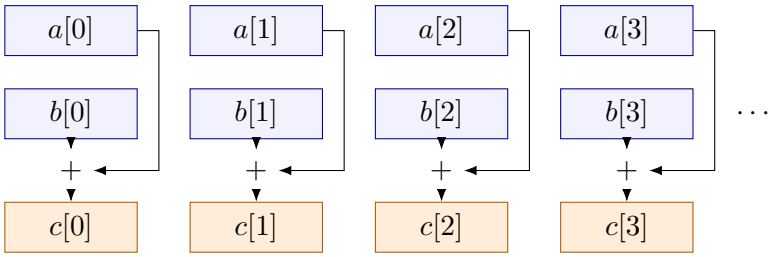


Figura 1: Adição elemento a elemento. A independência entre índices permite distribuir o laço na CPU ou em um dispositivo com offload.

O programa utiliza `float` e inicializa a_i com a conversão de i, b_i com duas vezes esse valor e o vetor de referência `res` com a mesma expressão de soma aplicada no cálculo. O vetor c começa zerado. Essa referência verifica a implementação da operação em precisão simples; não é uma solução independente em precisão maior.

2.2 Trabalho, memória e intensidade aritmética

Para $N = 10^7$ e `float` de quatro bytes, cada vetor ocupa 40 MB. Os quatro vetores do host ocupam 160 MB, sem contar bibliotecas e estruturas auxiliares. A soma percorre dois vetores de entrada e um de saída: são 120 MB de dados úteis por passagem.

Cada elemento envolve uma adição, duas leituras e uma escrita. A relação idealizada é

$$I = \frac{1 \text{ operação}}{12 \text{ bytes}} \approx 0{,}083 \text{ operação/byte}.$$

Essa baixa intensidade aritmética ajuda a explicar por que mover os dados pode custar mais que somá-los. A quantidade de núcleos ou a capacidade máxima de cálculo de uma GPU, isoladamente, não determina o tempo desse programa. Os 120 MB são uma contagem de dados do algoritmo; o tráfego físico pode ser diferente devido às caches e às políticas de escrita.

O acesso é contíguo: $a[i]$, $b[i]$ e $c[i]$ são percorridos em ordem. Isso favorece localidade espacial e operações vetoriais. Paralelismo entre threads e vetorização são mecanismos diferentes: o compilador pode processar vários elementos por instrução mesmo com uma única thread.

3 Organização e conceitos de OpenMP

3.1 Programa principal e funções auxiliares

O programa recebe `N`, `TOL` e `MODO`. O modo 0 chama `soma_cpu`; o modo 1 chama `soma_gpu`. As funções de `aux18.c` validam os argumentos, alocam e inicializam os vetores, verificam o resultado e liberam a memória.

Os arquivos `aux18.c` e `somas.c` são incluídos diretamente por `t18.c`. Portanto, o comando apresentado compila somente `t18.c`; os conteúdos dos outros dois arquivos já são incorporados ao mesmo módulo de compilação. Compilá-los novamente no mesmo comando provocaria definições duplicadas.

A alocação com `malloc` evita manter os quatro grandes vetores na pilha de uma função. Essa é uma adaptação útil em relação ao exemplo com arrays locais de tamanho fixo. Os arquivos não contêm um log de falha de pilha; por isso, a escolha é explicada como decisão de implementação, sem inventar um erro anterior.

3.2 CPU: parallel for

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

A diretiva cria uma região paralela e distribui as iterações entre as threads da equipe. Cada thread escreve em posições distintas de c. O fim da região sincroniza a equipe antes de o programa prosseguir. O número configurado vem de `OMP_NUM_THREADS`; os logs imprimem essa configuração, mas não medem o tamanho efetivo da equipe dentro da região.

No job de CPU, `OMP_PLACES=cores` descreve locais de execução. A configuração `OMP_PROC_BIND=close` solicita posicionar threads próximas nesses locais. Essas variáveis orientam o runtime; não acrescentam CPUs à alocação do Slurm nem comprovam, por si só, o posicionamento realizado.

3.3 GPU: target, loop e offload

O código enviado utiliza:

```
#pragma omp target
#pragma omp loop
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

`target` delimita uma região candidata à execução em um dispositivo. `loop` descreve um laço cujas iterações podem ser distribuídas no contexto de execução disponível. A tradução para a GPU depende de um compilador e de um runtime com suporte às diretivas e ao dispositivo [5].

Offload é o encaminhamento da execução do host para um dispositivo. *Host* é o sistema que inicia o programa, normalmente a CPU; *device* é o dispositivo de destino. Quando o runtime executa a região target no host em vez do acelerador, ocorre *fallback*. Assim, escrever `target` ou imprimir "GPU" não basta para comprovar offload.

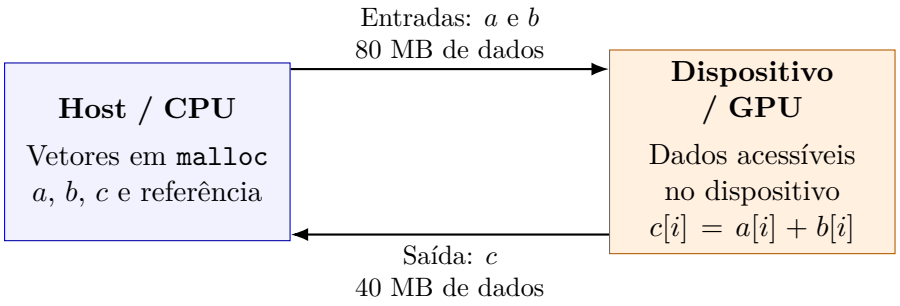
3.4 Ponteiros e acesso aos dados

No exemplo com arrays de tamanho conhecido, o compilador dispõe de informações sobre sua extensão. Nesta adaptação, a, b e c são ponteiros retornados por `malloc`. O valor do ponteiro identifica um endereço no host, mas não descreve sozinho todos os elementos do vetor nem garante que esse endereço seja acessível no dispositivo.

Para memórias separadas, uma forma explícita de indicar as regiões é:

```
map(to:a[0:N], b[0:N]) map(from:c[0:N])
```

`to` indica as entradas enviadas ao dispositivo; `from` indica o resultado que deve retornar ao host. O vetor de referência pode permanecer no host, onde será feita a verificação. Configurações de memória compartilhada ou gerenciada podem ter outro comportamento, mas não foram identificadas nos registros fornecidos.



`float`: quatro bytes por elemento; $N = 10^7$.
Copiar o valor de um ponteiro não equivale a copiar seu vetor.

Figura 2: Dados necessários para uma soma em memórias separadas. O desenho explica o mapeamento proposto; não representa transferências medidas nos logs históricos.

O tutorial adaptado possui exemplos com diferentes formas de alocação. Copiar apenas as diretivas de um exemplo não garante que o tratamento de dados continue equivalente após trocar arrays locais por ponteiros. A versão enviada não explicita o mapeamento das regiões de memória; esse ponto precisa ser resolvido e testado junto com a confirmação do dispositivo.

3.5 Validação e reduction

O programa conta quantas posições satisfazem

$$|c_i - \mathrm{res}_i| > \mathrm{TOL}, \qquad \mathrm{TOL} = 10^{-9}.$$

A verificação usa `parallel for reduction(+:err)`. Cada thread acumula sua contagem, e as contagens são somadas ao final. A redução evita que várias threads atualizem simultaneamente o mesmo contador sem coordenação.

Todos os registros mostraram `Erros: 0`. Isso confirma o atendimento ao critério implementado, mas não informa se a soma ocorreu na CPU ou na GPU. A tolerância é absoluta; como a referência usa a mesma operação em `float`, zero erros não significa precisão decimal de nove casas. O verificador também não testa explicitamente valores não finitos, e o `main` retorna sucesso mesmo se a contagem for positiva. Esses são limites da validação atual, não falhas numéricas observadas nos logs.

4 Ambiente, submissão e medição

4.1 Hardware disponível e hardware efetivamente utilizado

Os logs do modo CPU informam `service3.npad.internal`. Os do modo GPU informam `gpunode1`, com V100, e `gpunode4`, com H200. A relação de hardware do NPAD indicada e transcrita pelo autor descreve as classes da Tabela 1 [4].

Classe	Processadores do host	RAM do host	Aceleradores por nó
`gpu-8-v100`	2 Xeon E5-2683 v4	512 GB DDR4	8 V100-SXM2, 16 GB por GPU
`gpu-4-h200`	2 Xeon Gold 6548N	1 TB DDR5	4 H200, 141 GB HBM3e por GPU

Tabela 1: Especificações de referência das classes de nós, não uma coleta de hardware feita pelos jobs. RAM do host e memória da GPU são recursos distintos.

Um nó com GPU também tem CPUs. O `nvidia-smi` confirma a presença dos aceleradores visíveis; não comprova que o laço do programa foi executado neles. Além disso, os logs não registram o modelo de CPU de `service3`, a versão do compilador de cada série ou a associação exata entre todos os jobs e partições. Não se atribui a esse hostname o hardware de outro nó.

4.2 O que jobs.sh solicita

O script submete dois jobs independentes. O job de CPU pede a partição `intel-128`, um nó, uma tarefa e uma CPU por tarefa. O job de GPU pede `gpu-8-v100`, um nó, uma tarefa e quatro GPUs. Ambos têm limite de dois minutos e compilam com:

```
gcc -O3 -fopenmp -std=c11 t18.c -o executavel -lm
```

No job de GPU, exporta-se `OMP_TARGET_OFFLOAD=MANDATORY`. Em uma implementação que ofereça suporte a essa opção, ela exige o uso do dispositivo para regiões target não destinadas ao host, em vez de permitir fallback silencioso. Entretanto, a variável escrita no script não demonstra qual runtime foi usado ou que sua versão a reconheceu. É necessário verificar o comportamento real.

O script atual não reproduz sozinho todos os logs: ele configura uma thread de CPU, enquanto `CPU4.out` informa quatro, e solicita V100, enquanto um registro lista H200. Isso indica configurações diferentes entre as séries; os scripts históricos dessas

variações não foram fornecidos. Os executáveis enviados contêm identificação GCC 8.5.0, Red Hat `8.5.0-22`, mas isso não prova que todas as séries tenham sido geradas pelos mesmos binários.

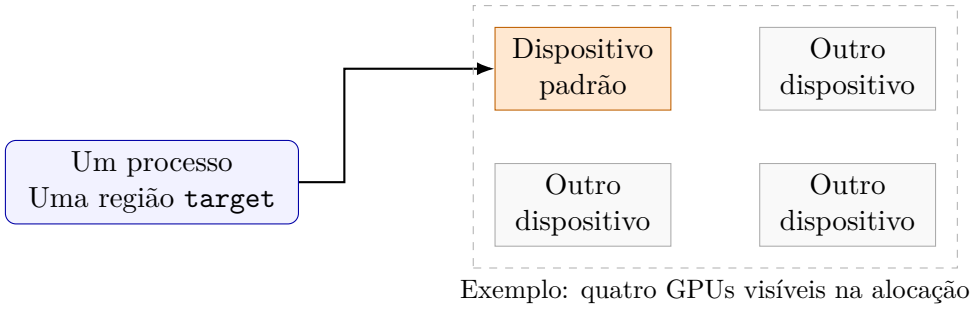
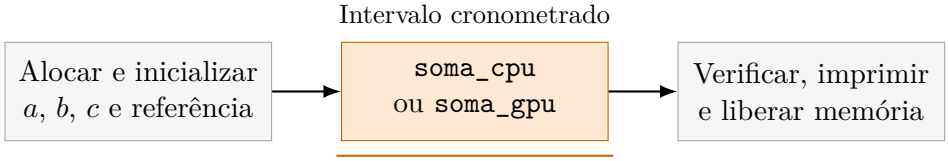


Figura 3: Reservar GPUs e utilizá-las são decisões distintas. Uma única região target, sem divisão explícita entre dispositivos, não distribui automaticamente o vetor por quatro GPUs. O desenho descreve o modelo, condicionado à existência de offload.

A região target não contém uma divisão do vetor entre dispositivos. Logo, nem o pedido de quatro GPUs nem quatro linhas no `nvidia-smi` permitem classificar uma execução como soma em quatro GPUs. Os registros são tratados como séries em nós com aceleradores visíveis, sem uma afirmação de paralelismo entre GPUs.

4.3 Intervalo cronometrado

`omp_get_wtime()` é chamado imediatamente antes e depois da função de soma. Alocação, inicialização, validação e impressão ficam fora do intervalo.



Em offload com transferência explícita, a chamada inclui envio, execução e retorno dos dados. Os logs não separam esses custos.

Figura 4: O tempo impresso mede somente a chamada de soma. Não inclui inicialização nem validação; a região target é síncrona no código apresentado.

Em uma execução verificada com mapeamento e memórias separadas, o tempo dessa chamada incluiria preparação do offload, transferências, execução e sincronização de retorno. Seria uma comparação da operação completa com dados inicialmente no host, não apenas do kernel residente na GPU. Os logs existentes não separam essas parcelas nem confirmam que ocorreram transferências.

4.4 Repetições e limite do aquecimento

Cada série contém cinco processos medidos. As quatro séries do modo GPU incluem ainda um processo anterior denominado `Warm-up`: são 35 medições e quatro aque-

cimentos, todos preservados. Os aquecimentos ficam fora das médias.

Cada comando `./t18_gpu` inicia um novo processo, que termina após uma soma. Portanto, o contexto e os recursos privados daquele processo não permanecem ativos no seguinte. A execução anterior pode influenciar condições gerais do sistema, mas não garante excluir o custo inicial do runtime ou da GPU das cinco medições. Um aquecimento do mesmo contexto exige repetição dentro do mesmo processo.

As médias e desvios são calculados separadamente para cada arquivo, sem remover amostras. Para cinco tempos T_r:

$$\overline{T} = \frac{1}{5}\sum_{r=1}^{5} T_r, \qquad s = \sqrt{\frac{\sum_{r=1}^{5}(T_r - \overline{T})^2}{4}}, \qquad CV = 100\frac{s}{\overline{T}}.$$

As barras dos gráficos representam um desvio padrão amostral, não intervalos de confiança. Os arquivos não contêm datas de execução para todas as séries nem estados finais de todos os jobs; esses dados não são inferidos dos nomes dos arquivos.

5 Resultados observados

5.1 Tempos e dispersão

Tabela 2: Cinco medições por série; aquecimentos separados e excluídos das médias. Os nomes de GPU identificam o hardware listado no nó, sem confirmar offload.

Série	Média (ms)	s (ms)	CV (%)	Aquecimento (ms)
CPU 1a	10,567663	0,019582	0,19	—
CPU 1b	9,974812	0,038587	0,39	—
CPU 4	5,379129	0,037736	0,70	—
V100 a	12,927817	0,127933	0,99	13,064327
V100 b	13,484205	0,339940	2,52	14,371583
H200	5,996597	0,042919	0,72	6,105013
V100 / 4 visíveis	12,175676	0,339729	2,79	12,091021

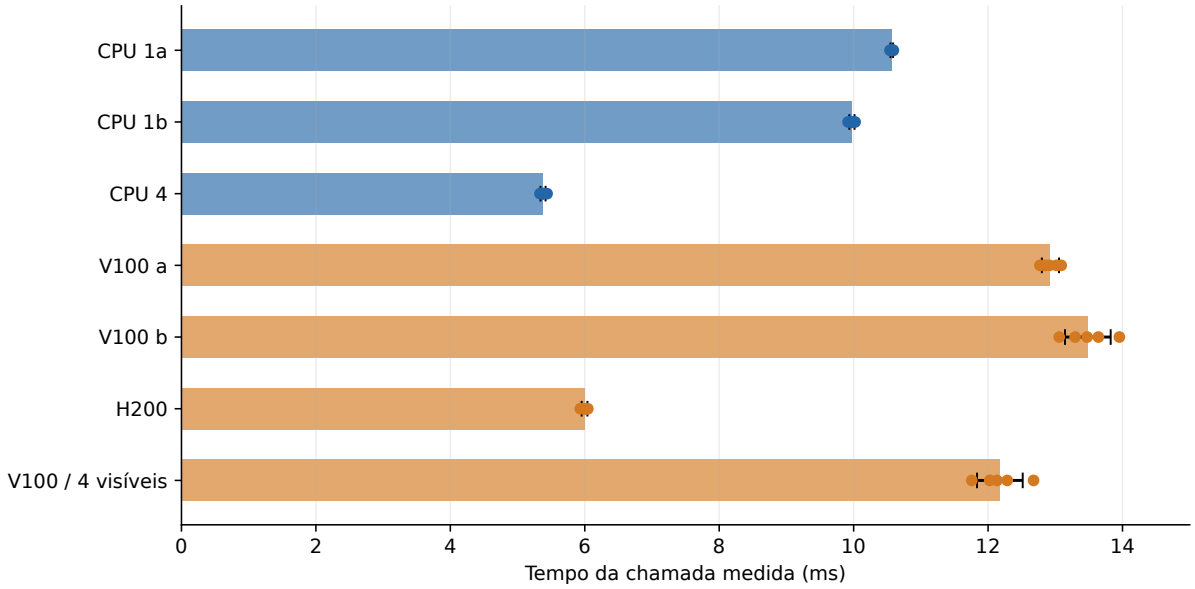


Figura 5: Todas as 35 medições, com médias e desvios. Azul: modo CPU. Laranja: modo declarado GPU, ainda sem comprovação do dispositivo de execução. V100 e H200 identificam o hardware listado nos nós.

Todas as séries têm dispersão interna relativamente pequena: os CVs variam de cerca de 0,19% a 2,79%. Isso indica repetibilidade dos tempos dentro dos arquivos, mas não comprova o dispositivo utilizado nem elimina diferenças entre campanhas.

As duas séries de uma thread diferem aproximadamente 5,94% em suas médias, mais que suas dispersões internas. Por isso, são mantidas separadas. Escolher apenas uma delas sem explicar a referência modificaria o ganho atribuído à execução com quatro threads.

5.2 Comparação entre uma e quatro threads na CPU

Definem-se a razão de tempos e a eficiência relativa por

$$S_4 = \frac{\overline{T}_1}{\overline{T}_4}, \qquad E_4 = \frac{S_4}{4}.$$

Tabela 3: Comparação CPU com o mesmo N. As duas referências permanecem separadas.

Referência de uma thread	T_1/T_4	E_4 (%)	T_4 (ms)
CPU 1a	1,9646	49,11	5,379129
CPU 1b	1,8544	46,36	5,379129

A soma com quatro threads configuradas levou cerca de 5,379 ms. O speedup fica entre 1,854 e 1,965, conforme a série de uma thread adotada, com eficiência entre 46,36% e 49,11%. Não é uma aceleração de quatro vezes, nem uma medição da porcentagem de

ocupação da CPU. As referências de uma thread também usam o executável OpenMP compilado com `-O3`.

A soma tem pouco cálculo por byte, de modo que as threads compartilham recursos de cache e memória ao percorrer os vetores. Também há custos de entrada na região paralela e sincronização. Esses mecanismos são compatíveis com ganho sublinear, mas os logs não possuem contadores ou perfis que permitam apontar uma causa exclusiva.

5.3 O que se pode dizer dos registros em nós com GPU

Entre as séries rotuladas GPU, a do nó que lista H200 teve a menor média, aproximadamente 5,997 ms; as séries V100 ficaram entre 12,176 e 13,484 ms. A série CPU com quatro threads foi a mais rápida entre os registros disponíveis.

Esses são resultados das chamadas medidas nos respectivos ambientes. Não permitem concluir que "a H200 somou mais rápido que a V100" ou que "a CPU venceu uma GPU efetivamente utilizada". Caso tenha ocorrido fallback, os tempos podem refletir o cálculo nas CPUs dos nós, que também são diferentes. Mesmo após confirmar offload, uma comparação entre nós distintos precisará considerar o host e as transferências.

Para uma soma isolada, existe apenas uma adição por elemento transferido. Mesmo uma GPU corretamente utilizada pode não compensar os custos de movimentação e lançamento nesse cenário. Essa é uma explicação conceitual para um resultado possível; não é uma decomposição comprovada dos tempos históricos.

6 Problemas identificados e encaminhamentos

6.1 Confirmar o local de execução

Os rótulos de `MODO` são escolhidos pelo argumento do programa. Não há chamada a `omp_is_initial_device()` nos registros. A inspeção estática do executável GPU encontrou uma função host de fallback e a chamada `GOMP_target_ext`, porém não identificou registro de uma imagem de dispositivo NVIDIA. A seção `.gnu.offload_funcs`, isoladamente, é uma tabela de funções e não comprova a existência dessa imagem. A inspeção integral está no apêndice.

Esses elementos são compatíveis com fallback, mas os binários não estão ligados a cada série por hashes registrados durante os jobs. A conclusão segura é que o uso efetivo da GPU permanece sem comprovação; os tempos não são apresentados como uma comparação final entre dispositivos.

O encaminhamento é registrar a quantidade de dispositivos com `omp_get_num_devices()`. Dentro da região target, avalia-se `omp_is_initial_device()`: retorno 1 indica host; retorno 0 indica execução fora do dispositivo inicial. O programa de diagnóstico do Apêndice C está preparado para essa verificação, mas não gerou os logs analisados. A confirmação deve ser feita também na região que realiza a soma.

6.2 Compatibilizar compilação e dados

O tutorial NPAD possui configurações específicas de compiladores para GPU. A configuração NVHPC, por exemplo, utiliza `nvc` com `-mp=gpu`; já `gcc -fopenmp` depende de como o GCC e o suporte a offload foram instalados. Não se escolhe uma arquitetura de compilação apenas pelo nome do arquivo executável.

Antes de repetir a comparação, é preciso registrar a versão e o comando completos do compilador, verificar suporte a `loop` e ao dispositivo e explicitar as regiões dos vetores ou documentar um modelo de memória compatível. As diretivas propostas não corrigem, sozinhas, uma instalação sem suporte a GPU. O relatório não afirma que essas mudanças já foram executadas no NPAD.

6.3 Distinguir decisões implementadas de correções pendentes

Ponto	O que existe	Situação ou encaminhamento
Vetores grandes	Alocação dinâmica e teste de falha	Implementado; evita depender de uma pilha grande.
Argumentos inválidos	Conversão com checagem de faixa e formato	Implementado para tamanho, tolerância e modo.
Contagem de erros	Redução paralela do contador	Implementado; zero erros em todos os logs.
Dispositivo de execução	Rótulo GPU e listagem de hardware	Confirmar dentro da região target.
Acesso aos vetores	Ponteiros de `malloc`, sem regiões explícitas	Especificar mapeamento ou modelo de memória válido.
Aquecimento	Uma execução anterior em processo separado	Repetir dentro do processo para aquecer o mesmo contexto.
Reprodução das séries	Script atual cobre apenas parte das configurações	Registrar scripts, ambiente, CPU, dispositivo e alocação por série.

Tabela 4: Progresso documentado e etapas necessárias para consolidar a comparação. Ausência de registro não é apresentada como solução já aplicada.

7 Conclusão

Foi implementado um programa de soma de vetores com modos CPU e target, entrada parametrizada, alocação dinâmica e verificação paralela. Os 39 registros apresentaram zero erros. Na comparação CPU, quatro threads configuradas reduziram o tempo de aproximadamente dez milissegundos para 5,379 ms, com aceleração próxima de duas vezes.

Os registros em nós com V100 e H200 permitem comparar os tempos observados, mas não demonstram que a soma executou no acelerador. A principal pendência para concluir a avaliação solicitada é confirmar o offload e o tratamento dos dados com uma

compilação compatível, mantendo os custos medidos claramente definidos. Reservar uma GPU, listar seu modelo e imprimir o modo GPU são informações diferentes de verificar o dispositivo de execução.

A atividade mostra que correção numérica, paralelismo e desempenho precisam ser verificados separadamente. Em um algoritmo de baixa intensidade aritmética, a movimentação de dados e os custos de execução podem ser decisivos. O relatório documenta o progresso e fornece uma base reproduzível para completar essa confirmação, sem atribuir aos registros resultados que eles não comprovam.

A Requisitos e proveniência

A.1 Enunciado completo

```
Nome da Tarefa:
Tarefa 18: Comparando soma de vetores entre CPU e GPU em OpenMP
Descrição:

Faça os exercícios de adição de vetores, vadd.c, dos slides 27 e 48 do tutorial de programação de
    GPUs com OpenMP em um dos nós com GPU do NPAD. Compare os tempos de execução somente na CPU e
    com o uso da GPU. Reporte seu progresso apresentando os problemas encontrados e as soluções
    apresentadas.

Tutorial sobre programação de GPUs com OpenMP da Universidade de Bristol:

https://github.com/UoB-HPC/openmp-tutorial

https://github.com/NPAD-UFRN/openmp-tutorial (adaptação para o NPAD)
```

A.2 Proveniência dos arquivos

```
Fontes e resultados: commit e17b7d07f4e0ae1a3f1cdd65e0a8f587638e0bbf
Disponiveis em main na revisao 9118003, consultada em 18/10/2026.
Datas e versoes exatas das execucoes: nao registradas em todos os logs.
Tutorial NPAD consultado: revisao a8ceec45357cd71594f569aed98ed631e9461a8b.
Inspecao de binarios: leitura estatica local, sem executar as simulacoes.
Os binarios nao foram recompilados para gerar os dados do relatorio.
```

A.3 Referência de hardware utilizada

```
Fonte indicada pelo autor: https://npad.ufrn.br/npad/hardware
Transcricao relevante fornecida pelo autor em 18/10/2026.
A pagina nao pode ser recuperada automaticamente nesta preparacao.
As especificacoes abaixo descrevem classes de nos, nao uma medicao do job.

Nos GPU Tesla V100 / particao gpu-8-v100:
  2 nos; por no:
  2 x Intel Xeon E5-2683 v4
  512 GB DDR4 (memoria do host)
  8 x NVIDIA Tesla V100-SXM2 de 16 GB (memoria por GPU)
  SSD local de 1,92 TB
  InfiniBand ConnectX-4 FDR 56 Gb/s

Nos GPU H200 / particao gpu-4-h200:
  2 nos; por no:
  2 x Intel Xeon Gold 6548N
  1 TB DDR5 (memoria do host)
  4 x NVIDIA H200 com 141 GB HBM3e (memoria por GPU)
  4 x SSD NVMe de 1,5 TB em RAID0
  InfiniBand ConnectX-6 HDR 100 Gb/s

Os logs identificam V100 em gpunode1 e H200 em gpunode4.
Os modelos de CPU acima nao foram medidos ou impressos por esses jobs.
O hostname CPU service3 nao foi associado a uma classe de hardware pelos logs.
```

B Códigos completos da implementação analisada

Os arquivos são incluídos por `\lstinputlisting`, sem cortes de linhas. São cópias integrais dos fontes enviados, incluindo as limitações discutidas no corpo do relatório. Não foram substituídos por uma implementação corrigida ao preparar as figuras.

B.1 Programa principal: t18.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#include "aux18.c"
#include "somas.c"

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "Uso: %s <N> <TOL> <MODO>\n", argv[0]);
        fprintf(stderr, "  N    = tamanho dos vetores\n");
        fprintf(stderr, "  TOL  = tolerancia\n");
        fprintf(stderr, "  MODO = 0 (CPU) ou 1 (GPU)\n");

        return EXIT_FAILURE;
    }

    // Leitura e validacao dos argumentos
    int N = ler_n(argv[1]);
    float TOL = ler_tol(argv[2]);
    int modo = ler_modo(argv[3]);

    // Alocacao
    float *a = aloca_vetor(N);
    float *b = aloca_vetor(N);
    float *c = aloca_vetor(N);
    float *res = aloca_vetor(N);

    // Inicializacao
    inicializa_vetores(a, b, c, res, N);

    // Execucao
    double inicio = omp_get_wtime();

    if (modo == 0) {
        soma_cpu(a, b, c, N);
    } else {
        soma_gpu(a, b, c, N);
    }

    double fim = omp_get_wtime();

    // Validacao
    int err = verifica_resultado(c, res, N, TOL);

    // Resultado
    printf("====================================\n");
    printf("Soma de vetores\n");
    printf("N      : %d\n", N);
    printf("TOL    : %e\n", TOL);
    printf("MODO   : %s\n", modo == 0 ? "CPU" : "GPU");
    printf("Erros  : %d\n", err);
    printf("Tempo  : %.9f s\n", fim - inicio);
    printf("====================================\n");

    // Liberacao
```

```
58      libera_vetores(a, b, c, res);
59
60      return EXIT_SUCCESS;
61  }
```

B.2 Somas: somas.c

```c
#include <stdlib.h>
#include <omp.h>

void soma_cpu(float *a, float *b, float *c, int N)
{
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}

void soma_gpu(float *a, float *b, float *c, int N)
{
    #pragma omp target
    #pragma omp loop
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}
```

B.3 Funções auxiliares: aux18.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include <math.h>

int ler_n(const char *arg)
{
    char *end;
    long valor;

    errno = 0;
    valor = strtol(arg, &end, 10);

    if (end == arg || *end != '\0') {
        fprintf(stderr, "Erro: N invalido: '%s'\n", arg);
        exit(EXIT_FAILURE);
    }

    if (errno == ERANGE || valor > INT_MAX || valor < INT_MIN) {
        fprintf(stderr, "Erro: N fora do intervalo permitido: '%s'\n", arg);
        exit(EXIT_FAILURE);
    }

    if (valor <= 0) {
        fprintf(stderr, "Erro: N deve ser maior que zero.\n");
        exit(EXIT_FAILURE);
    }

    return (int)valor;
}


float ler_tol(const char *arg)
{
    char *end;
    float valor;

    errno = 0;
    valor = strtof(arg, &end);

    if (end == arg || *end != '\0') {
```

```
43          fprintf(stderr, "Erro: TOL invalida: '%s'\n", arg);
44          exit(EXIT_FAILURE);
45      }
46
47      if (errno == ERANGE) {
48          fprintf(stderr, "Erro: TOL fora do intervalo permitido: '%s'\n", arg);
49          exit(EXIT_FAILURE);
50      }
51
52      if (!isfinite(valor) || valor < 0.0f) {
53          fprintf(stderr, "Erro: TOL deve ser um valor finito maior ou igual a zero.\n");
54          exit(EXIT_FAILURE);
55      }
56
57      return valor;
58  }
59
60
61  int ler_modo(const char *arg)
62  {
63      char *end;
64      long valor;
65
66      errno = 0;
67      valor = strtol(arg, &end, 10);
68
69      if (end == arg || *end != '\0') {
70          fprintf(stderr, "Erro: MODO invalido: '%s'\n", arg);
71          exit(EXIT_FAILURE);
72      }
73
74      if (errno == ERANGE || valor < INT_MIN || valor > INT_MAX) {
75          fprintf(stderr, "Erro: MODO invalido: '%s'\n", arg);
76          exit(EXIT_FAILURE);
77      }
78
79      if (valor != 0 && valor != 1) {
80          fprintf(stderr, "Erro: MODO deve ser 0 (CPU) ou 1 (GPU).\n");
81          exit(EXIT_FAILURE);
82      }
83
84      return (int)valor;
85  }
86
87
88  float *aloca_vetor(int N)
89  {
90      float *vetor = malloc((size_t)N * sizeof(float));
91
92      if (vetor == NULL) {
93          fprintf(stderr, "Erro: falha na alocacao de memoria.\n");
94          exit(EXIT_FAILURE);
95      }
96
97      return vetor;
98  }
99
100
101 void inicializa_vetores(float *a, float *b, float *c, float *res, int N)
102 {
103     for (int i = 0; i < N; i++) {
104         a[i] = (float)i;
105         b[i] = 2.0f * (float)i;
106         c[i] = 0.0f;
107         res[i] = a[i] + b[i];
108     }
109 }
110
111
112 int verifica_resultado(float *c, float *res, int N, float TOL)
```

```
113 {
114     int err = 0;
115 
116     #pragma omp parallel for reduction(+:err)
117     for (int i = 0; i < N; i++) {
118         float diff = c[i] - res[i];
119 
120         if (fabsf(diff) > TOL) {
121             err++;
122         }
123     }
124 
125     return err;
126 }
127 
128 
129 void libera_vetores(float *a, float *b, float *c, float *res)
130 {
131     free(a);
132     free(b);
133     free(c);
134     free(res);
135 }
```

B.4 Submissão: jobs.sh

```
#!/bin/bash

# ============================================================
# Job CPU
# ============================================================

sbatch <<'EOF'
#!/bin/bash

#SBATCH --partition=intel-128
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --time=00:02:00
#SBATCH --job-name=ppCPU_job
#SBATCH --output=resultado_cpu_%j.out

cd "$SLURM_SUBMIT_DIR"

gcc -O3 -fopenmp -std=c11 t18.c -o t18_cpu -lm

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PLACES=cores
export OMP_PROC_BIND=close

echo "======================================"
echo "EXECUCAO CPU"
echo "Hostname: $(hostname)"
echo "Threads OpenMP: $OMP_NUM_THREADS"
echo "======================================"
echo

for i in {1..5}; do
    echo "----- Execucao $i -----"
    ./t18_cpu 10000000 1e-9 0
    echo
done
EOF


# ============================================================
# Job GPU
# ============================================================

sbatch <<'EOF'
#!/bin/bash

#SBATCH --partition=gpu-8-v100
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:4
#SBATCH --gres-flags=enforce-binding
#SBATCH --time=00:02:00
#SBATCH --job-name=ppGPU_job
#SBATCH --output=resultado_gpu_%j.out

cd "$SLURM_SUBMIT_DIR"

export OMP_TARGET_OFFLOAD=MANDATORY

gcc -O3 -fopenmp -std=c11 t18.c -o t18_gpu -lm

echo "======================================"
echo "EXECUCAO GPU"
echo "Hostname: $(hostname)"
echo "======================================"

```

```
nvidia-smi --query-gpu=name --format=csv,noheader
echo

echo "----- Warm-up -----"
./t18_gpu 10000000 1e-9 1
echo

for i in {1..5}; do
    echo "----- Execucao $i -----"
    ./t18_gpu 10000000 1e-9 1
    echo
done
EOF
```

C Diagnóstico proposto para uma nova execução

O programa abaixo foi preparado como apoio à continuidade e não foi executado no NPAD para gerar este relatório. Deve ser compilado com suporte a offload para o dispositivo disponível e executado dentro de uma alocação. Ele termina com falha se não encontrar dispositivo ou se a região executar no host. Um resultado positivo verifica essa região de diagnóstico; a soma e seu tratamento de dados ainda precisam da mesma confirmação.

```c
/* Apoio para uma nova execucao: nao gerou os resultados historicos. */
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int devices = omp_get_num_devices();
    int on_host = 1;
    printf("_OPENMP: %d\n", _OPENMP);
    printf("Dispositivos OpenMP: %d\n", devices);
    if (devices < 1) {
        fprintf(stderr, "Nenhum dispositivo OpenMP disponivel.\n");
        return EXIT_FAILURE;
    }

    #pragma omp target map(from:on_host)
    {
        on_host = omp_is_initial_device();
    }
    printf("Executou no host: %d\n", on_host);
    if (on_host) {
        fprintf(stderr, "A regiao target executou no host.\n");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

D Medições completas

Os valores são extraídos automaticamente dos arquivos de saída. Aquecimento é a repetição zero. Os nomes das séries preservam a distinção entre arquivos; os nomes de GPU não confirmam offload.

Tabela 5: Todos os 39 registros. Repetição 0 identifica o aquecimento em processo separado.

Série	Repetição	Tempo (s)	Erros
CPU 1a	1	0,010591173	0
CPU 1a	2	0,010564717	0
CPU 1a	3	0,010541467	0
CPU 1a	4	0,010558852	0
CPU 1a	5	0,010582105	0
CPU 1b	1	0,009982172	0
CPU 1b	2	0,010022212	0
CPU 1b	3	0,009961779	0
CPU 1b	4	0,009917792	0
CPU 1b	5	0,009990105	0
CPU 4	1	0,005374323	0
CPU 4	2	0,005372279	0
CPU 4	3	0,005439168	0
CPU 4	4	0,005334313	0
CPU 4	5	0,005375561	0
V100 a	0	0,013064327	0
V100 a	1	0,013018875	0
V100 a	2	0,012913231	0
V100 a	3	0,013088888	0
V100 a	4	0,012771590	0
V100 a	5	0,012846501	0
V100 b	0	0,014371583	0
V100 b	1	0,013473431	0
V100 b	2	0,013641344	0
V100 b	3	0,013953403	0
V100 b	4	0,013058480	0
V100 b	5	0,013294365	0
H200	0	0,006105013	0
H200	1	0,006026457	0
H200	2	0,005933619	0
H200	3	0,005993496	0
H200	4	0,005984337	0
H200	5	0,006045078	0
V100 / 4 visíveis	0	0,012091021	0
V100 / 4 visíveis	1	0,012676509	0
V100 / 4 visíveis	2	0,012025466	0
V100 / 4 visíveis	3	0,012285496	0
V100 / 4 visíveis	4	0,011758101	0
V100 / 4 visíveis	5	0,012132809	0

D.1 Hashes dos fontes, logs e binários recebidos

Os hashes identificam os arquivos recebidos no GitHub, não uma associação retrospectiva comprovada entre cada binário e cada execução.

```
dafcab4a64266d3d45630b2d53a4cbc8ceb445909604353d82016291359d0ada  CPU1a.out
fa7d39550a34cc2209fc7fde3249da3dfdc8ac4defceb3022c9a85a8006f9387  CPU1b.out
d41b8c82eb0c5be135c565331cab434dcb976847b1a7e4f5fa257e55dc643d21  CPU4.out
e8347f8056edac440eebc7c9bcf6067527baadfa874729b32768e4bc38acabda  GPUa.out
ab0ded7015290506cf4c34204c342ea24cfd7473aac274ba6df65848d793458f  GPUb.out
b29a394a8574e3cde627484c2eaa9062f86d2686f340260746c593f16b597876  aux18.c
e74a5034251b1b880a672a3b1d9ed083cd67670d6d01ab19a6f9cf771d5d2ce9  jobs.sh
20f27067c758831623b8d2aa12fab74885f446e1abbe698331c44afc9d43b6a5  requisitosTarefa18.txt
08a87e2b74d36f24c0974a787d6e8ab6be0decb0df59ba0ef821f7e9b5b60e1d  resultado_gpu_2133025.out
89757b8df5ca845c27b529a6f40f12a1a9f2f5294e82ebaf706991b759a30d01  resultado_gpu_2134904.out
7de66807492db57081c4c1eb79bc61d50750108f57a40d658989851e1000ae11  somas.c
e5d343c02fdd9cbb33270056119140b679a5c9451010a71d49f19f43969f8bbc  t18.c
b09553da79f710bd90f1b082f2e2e369d9b33adc6e08f1be00fdbf8776a4ec96  t18_cpu
cb72b3652e80d96c1a68d5a2074f8bfa97d725cd79231d31d8a20ad4e8b69139  t18_gpu
```

E Registros integrais das execuções

E.1 CPU1a.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 1
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010591173 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010564717 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010541467 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010558852 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010582105 s
==================================
```

E.2 CPU1b.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 1
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009982172 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.010022212 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009961779 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009917792 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.009990105 s
==================================
```

E.3 CPU4.out

```
====================================
EXECUCAO CPU
Hostname: service3.npad.internal
Threads OpenMP: 4
====================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005374323 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005372279 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005439168 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005334313 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : CPU
Erros  : 0
Tempo  : 0.005375561 s
==================================
```

E.4 GPUa.out

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013064327 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013018875 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012913231 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013088888 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012771590 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012846501 s
==================================
```

E.5 GPUb.out

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.014371583 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013473431 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013641344 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013953403 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013058480 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.013294365 s
==================================
```

E.6 resultado_gpu_2133025.out

```
====================================
EXECUCAO GPU
Hostname: gpunode4.npad.internal
====================================
NVIDIA H200

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.006105013 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.006026457 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.005933619 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.005993496 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.005984337 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.006045078 s
==================================
```

E.7 resultado_gpu_2134904.out

```
====================================
EXECUCAO GPU
Hostname: gpunode1.npad.internal
====================================
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB
Tesla V100-SXM2-16GB

----- Warm-up -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012091021 s
==================================

----- Execucao 1 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012676509 s
==================================

----- Execucao 2 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012025466 s
==================================

----- Execucao 3 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012285496 s
==================================

----- Execucao 4 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.011758101 s
==================================

----- Execucao 5 -----
==================================
Soma de vetores
N      : 10000000
TOL    : 1.000000e-09
MODO   : GPU
Erros  : 0
Tempo  : 0.012132809 s
```

```
==================================
```

F Inspeção estática do executável recebido

Os comandos abaixo leem os binários, sem executar a soma. A identificação de código x86-64 do host é normal em executáveis com offload; o ponto discutido é a ausência de evidência de registro de imagem de dispositivo neste material. Uma tabela de funções offload, sozinha, não identifica o local de execução de um log histórico.

```
$ file Tarefa_18/t18_cpu Tarefa_18/t18_gpu
Tarefa_18/t18_cpu: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
     interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=8
     af81fa640841fc8951fb5a9396db04e94137b3b, not stripped
Tarefa_18/t18_gpu: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
     interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=
     cfa9a930bf02fdeb6b4aebfdbe53c7ad4e51b65d, not stripped

$ readelf -p .comment Tarefa_18/t18_gpu

String dump of section '.comment':
  [     0]  GCC: (GNU) 8.5.0 20210514 (Red Hat 8.5.0-22)


$ readelf -SW Tarefa_18/t18_gpu
There are 31 section headers, starting at offset 0x6638:

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        0000000000400238 000238 00001c 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE            0000000000400254 000254 000020 00   A  0   0  4
  [ 3] .note.gnu.build-id NOTE            0000000000400274 000274 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        0000000000400298 000298 000024 00   A  5   0  8
  [ 5] .dynsym           DYNSYM          00000000004002c0 0002c0 0001f8 18   A  6   1  8
  [ 6] .dynstr           STRTAB          00000000004004b8 0004b8 000155 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          000000000040060e 00060e 00002a 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         0000000000400638 000638 000090 00   A  6   3  8
  [ 9] .rela.dyn         RELA            00000000004006c8 0006c8 000078 18   A  5   0  8
  [10] .rela.plt         RELA            0000000000400740 000740 000168 18  AI  5  22  8
  [11] .init             PROGBITS        00000000004008a8 0008a8 00001b 00  AX  0   0  4
  [12] .plt              PROGBITS        00000000004008d0 0008d0 000100 10  AX  0   0 16
  [13] .text             PROGBITS        00000000004009d0 0009d0 000ed5 00  AX  0   0 16
  [14] .fini             PROGBITS        00000000004018a8 0018a8 00000d 00  AX  0   0  4
  [15] .rodata           PROGBITS        00000000004018c0 0018c0 0002c4 00   A  0   0 16
  [16] .eh_frame_hdr     PROGBITS        0000000000401b84 001b84 00009c 00   A  0   0  4
  [17] .eh_frame         PROGBITS        0000000000401c20 001c20 000300 00   A  0   0  8
  [18] .init_array       INIT_ARRAY      0000000000602dd0 002dd0 000008 08  WA  0   0  8
  [19] .fini_array       FINI_ARRAY      0000000000602dd8 002dd8 000008 08  WA  0   0  8
  [20] .dynamic          DYNAMIC         0000000000602de0 002de0 000200 10  WA  6   0  8
  [21] .got              PROGBITS        0000000000602fe0 002fe0 000020 08  WA  0   0  8
  [22] .got.plt          PROGBITS        0000000000603000 003000 000090 08  WA  0   0  8
  [23] .data             PROGBITS        0000000000603090 003090 000010 00  WA  0   0  8
  [24] .gnu.offload_funcs PROGBITS        00000000006030a0 0030a0 000008 00  WA  0   0  8
  [25] .bss              NOBITS          00000000006030c0 0030a8 000048 00  WA  0   0 32
  [26] .comment          PROGBITS        0000000000000000 0030a8 00002d 01  MS  0   0  1
  [27] .gnu.build.attributes NOTE            0000000000a03108 0030d8 001e3c 00      0   0  4
  [28] .symtab           SYMTAB          0000000000000000 004f18 000d20 18     29  91  8
  [29] .strtab           STRTAB          0000000000000000 005c38 0008d3 00      0   0  1
  [30] .shstrtab         STRTAB          0000000000000000 00650b 00012c 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  D (mbind), l (large), p (processor specific)

$ nm -a Tarefa_18/t18_gpu
0000000000000000 a
00000000004018a5 t .annobin___libc_csu_fini.end
0000000000401895 t .annobin___libc_csu_fini.start
```

```
0000000000401895 t .annobin___libc_csu_init.end
0000000000401830 t .annobin___libc_csu_init.start
0000000000400de5 t .annobin__dl_relocate_static_pie.end
0000000000400de0 t .annobin__dl_relocate_static_pie.start
0000000000401830 t .annobin_elf_init.c
00000000004009d0 t .annobin_elf_init.c.exit
0000000000400dac t .annobin_elf_init.c.hot
0000000000400dac t .annobin_elf_init.c.startup
00000000004009d0 t .annobin_elf_init.c.unlikely
00000000004018a5 t .annobin_elf_init.c_end
00000000004009d0 t .annobin_elf_init.c_end.exit
0000000000400dac t .annobin_elf_init.c_end.hot
0000000000400dac t .annobin_elf_init.c_end.startup
00000000004009d0 t .annobin_elf_init.c_end.unlikely
0000000000400ddf t .annobin_init.c
00000000004009d0 t .annobin_init.c.exit
0000000000400dac t .annobin_init.c.hot
00000000004009d0 t .annobin_init.c.startup
00000000004009d0 t .annobin_init.c.unlikely
0000000000400ddf t .annobin_init.c_end
00000000004009d0 t .annobin_init.c_end.exit
0000000000400dac t .annobin_init.c_end.hot
00000000004009d0 t .annobin_init.c_end.startup
00000000004009d0 t .annobin_init.c_end.unlikely
0000000000400de0 t .annobin_static_reloc.c
00000000004009d0 t .annobin_static_reloc.c.exit
0000000000400dac t .annobin_static_reloc.c.hot
00000000004009d0 t .annobin_static_reloc.c.startup
00000000004009d0 t .annobin_static_reloc.c.unlikely
0000000000400ddf t .annobin_static_reloc.c_end
00000000004009d0 t .annobin_static_reloc.c_end.exit
0000000000400dac t .annobin_static_reloc.c_end.hot
00000000004009d0 t .annobin_static_reloc.c_end.startup
00000000004009d0 t .annobin_static_reloc.c_end.unlikely
00000000006030c0 b .bss
0000000000000000 n .comment
0000000000603090 d .data
0000000000602de0 d .dynamic
00000000004004b8 r .dynstr
00000000004002c0 r .dynsym
0000000000401c20 r .eh_frame
0000000000401b84 r .eh_frame_hdr
00000000004018a8 t .fini
0000000000602dd8 d .fini_array
0000000000a03108 n .gnu.build.attributes
0000000000400298 r .gnu.hash
00000000006030a0 d .gnu.offload_funcs
000000000040060e r .gnu.version
0000000000400638 r .gnu.version_r
0000000000602fe0 d .got
0000000000603000 d .got.plt
00000000004008a8 t .init
0000000000602dd0 d .init_array
0000000000400238 r .interp
0000000000400254 r .note.ABI-tag
0000000000400274 r .note.gnu.build-id
00000000006030a0 d .offload_func_table
0000000000603098 d .omp_data_kinds.33.3659
00000000006030e0 b .omp_data_sizes.32.3658
00000000004008d0 t .plt
00000000004006c8 r .rela.dyn
0000000000400740 r .rela.plt
00000000004018c0 r .rodata
00000000004009d0 t .text
0000000000000000 a /usr/lib/gcc/x86_64-redhat-linux/8/../../../../lib64/crt1.o
                 U GOMP_parallel@@GOMP_4.0
                 U GOMP_target_ext@@GOMP_4.5
0000000000602de0 d _DYNAMIC
0000000000603000 d _GLOBAL_OFFSET_TABLE_
00000000004018c0 R _IO_stdin_used
```

```
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
0000000000401f1c r __FRAME_END__
0000000000401b84 r __GNU_EH_FRAME_HDR
00000000006030a0 D __TMC_END__
00000000006030a8 B __bss_start
0000000000603090 D __data_start
0000000000400e60 t __do_global_dtors_aux
0000000000602dd8 d __do_global_dtors_aux_fini_array_entry
00000000004018c8 R __dso_handle
                 U __errno_location@@GLIBC_2.2.5
0000000000602dd0 d __frame_dummy_init_array_entry
                 w __gmon_start__
0000000000603100 B __gnu_lto_v1
0000000000602dd8 d __init_array_end
0000000000602dd0 d __init_array_start
00000000004018a0 T __libc_csu_fini
0000000000401830 T __libc_csu_init
                 U __libc_start_main@@GLIBC_2.2.5
00000000006030a0 D __offload_func_table
00000000006030a8 D __offload_funcs_end
00000000006030a8 D __offload_var_table
00000000006030a8 D __offload_vars_end
0000000000400de0 T _dl_relocate_static_pie
00000000006030a8 D _edata
0000000000603108 B _end
00000000004018a8 T _fini
00000000004008a8 T _init
0000000000400db0 T _start
0000000000401480 T aloca_vetor
00000000006030c8 b completed.7303
0000000000000000 a crtstuff.c
0000000000000000 a crtstuff.c
0000000000603090 W data_start
0000000000400df0 t deregister_tm_clones
0000000000000000 a elf-init.oS
                 U exit@@GLIBC_2.2.5
                 U fprintf@@GLIBC_2.2.5
0000000000400e90 t frame_dummy
                 U free@@GLIBC_2.2.5
                 U fwrite@@GLIBC_2.2.5
00000000004014c0 T inicializa_vetores
00000000004013d0 T ler_modo
0000000000401270 T ler_n
0000000000401320 T ler_tol
0000000000401750 T libera_vetores
00000000004009d0 T main
                 U malloc@@GLIBC_2.2.5
0000000000000000 a offloadstuff.c
0000000000000000 a offloadstuff.c
                 U omp_get_num_threads@@OMP_1.0
                 U omp_get_thread_num@@OMP_1.0
                 U omp_get_wtime@@OMP_2.0
                 U printf@@GLIBC_2.2.5
                 U puts@@GLIBC_2.2.5
0000000000400e20 t register_tm_clones
0000000000401780 T soma_cpu
0000000000401000 t soma_cpu._omp_fn.1
00000000004017c0 T soma_gpu
0000000000401160 t soma_gpu._omp_fn.2
00000000006030c0 B stderr@@GLIBC_2.2.5
                 U strtof@@GLIBC_2.2.5
                 U strtol@@GLIBC_2.2.5
0000000000000000 a t18.c
0000000000401700 T verifica_resultado
0000000000400ea0 t verifica_resultado._omp_fn.0

$ objdump -d --disassemble=soma_gpu Tarefa_18/t18_gpu

Tarefa_18/t18_gpu:     file format elf64-x86-64
```

```
Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .text:

00000000004017c0 <soma_gpu>:
  4017c0:   48 83 ec 50             sub    $0x50,%rsp
  4017c4:   48 63 c9                movslq %ecx,%rcx
  4017c7:   66 0f 6f 05 a1 03 00    movdqa 0x3a1(%rip),%xmm0        # 401b70 <__dso_handle+0x2a8>
  4017ce:   00
  4017cf:   41 b9 98 30 60 00       mov    $0x603098,%r9d
  4017d5:   48 8d 44 24 08          lea    0x8(%rsp),%rax
  4017da:   48 89 4c 24 28          mov    %rcx,0x28(%rsp)
  4017df:   41 b8 e0 30 60 00       mov    $0x6030e0,%r8d
  4017e5:   48 89 54 24 30          mov    %rdx,0x30(%rsp)
  4017ea:   ba 04 00 00 00          mov    $0x4,%edx
  4017ef:   48 89 74 24 38          mov    %rsi,0x38(%rsp)
  4017f4:   be 60 11 40 00          mov    $0x401160,%esi
  4017f9:   48 89 7c 24 40          mov    %rdi,0x40(%rsp)
  4017fe:   bf ff ff ff ff          mov    $0xffffffff,%edi
  401803:   0f 29 44 24 08          movaps %xmm0,0x8(%rsp)
  401808:   48 c7 44 24 18 00 00    movq   $0x0,0x18(%rsp)
  40180f:   00 00
  401811:   50                      push   %rax
  401812:   6a 00                   push   $0x0
  401814:   6a 00                   push   $0x0
  401816:   48 8d 4c 24 40          lea    0x40(%rsp),%rcx
  40181b:   e8 00 f1 ff ff          call   400920 <GOMP_target_ext@plt>
  401820:   48 83 c4 68             add    $0x68,%rsp
  401824:   c3                      ret

Disassembly of section .fini:

$ objdump -d --disassemble=soma_gpu._omp_fn.2 Tarefa_18/t18_gpu

Tarefa_18/t18_gpu:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .text:

0000000000401160 <soma_gpu._omp_fn.2>:
  401160:   4c 8b 07                mov    (%rdi),%r8
  401163:   48 8b 57 08             mov    0x8(%rdi),%rdx
  401167:   48 8b 4f 10             mov    0x10(%rdi),%rcx
  40116b:   48 8b 77 18             mov    0x18(%rdi),%rsi
  40116f:   45 85 c0                test   %r8d,%r8d
  401172:   0f 8e f3 00 00 00       jle    40126b <soma_gpu._omp_fn.2+0x10b>
  401178:   48 8d 79 10             lea    0x10(%rcx),%rdi
  40117c:   4c 8d 52 10             lea    0x10(%rdx),%r10
  401180:   48 39 fa                cmp    %rdi,%rdx
  401183:   41 8d 40 ff             lea    -0x1(%r8),%eax
  401187:   41 0f 93 c1             setae  %r9b
  40118b:   4c 39 d1                cmp    %r10,%rcx
  40118e:   40 0f 93 c7             setae  %dil
  401192:   41 09 f9                or     %edi,%r9d
  401195:   48 8d 7e 10             lea    0x10(%rsi),%rdi
  401199:   48 39 fa                cmp    %rdi,%rdx
  40119c:   40 0f 93 c7             setae  %dil
  4011a0:   4c 39 d6                cmp    %r10,%rsi
  4011a3:   41 0f 93 c2             setae  %r10b
  4011a7:   44 09 d7                or     %r10d,%edi
  4011aa:   41 84 f9                test   %dil,%r9b
  4011ad:   0f 84 8d 00 00 00       je     401240 <soma_gpu._omp_fn.2+0xe0>
```

```
  4011b3:   83 f8 02                cmp    $0x2,%eax
  4011b6:   0f 86 84 00 00 00       jbe    401240 <soma_gpu._omp_fn.2+0xe0>
  4011bc:   44 89 c7                mov    %r8d,%edi
  4011bf:   31 c0                   xor    %eax,%eax
  4011c1:   c1 ef 02                shr    $0x2,%edi
  4011c4:   48 c1 e7 04             shl    $0x4,%rdi
  4011c8:   0f 1f 84 00 00 00 00    nopl   0x0(%rax,%rax,1)
  4011cf:   00
  4011d0:   0f 10 04 06             movups (%rsi,%rax,1),%xmm0
  4011d4:   0f 10 0c 01             movups (%rcx,%rax,1),%xmm1
  4011d8:   0f 58 c1                addps  %xmm1,%xmm0
  4011db:   0f 11 04 02             movups %xmm0,(%rdx,%rax,1)
  4011df:   48 83 c0 10             add    $0x10,%rax
  4011e3:   48 39 f8                cmp    %rdi,%rax
  4011e6:   75 e8                   jne    4011d0 <soma_gpu._omp_fn.2+0x70>
  4011e8:   44 89 c0                mov    %r8d,%eax
  4011eb:   83 e0 fc                and    $0xfffffffc,%eax
  4011ee:   41 39 c0                cmp    %eax,%r8d
  4011f1:   74 78                   je     40126b <soma_gpu._omp_fn.2+0x10b>
  4011f3:   48 63 f8                movslq %eax,%rdi
  4011f6:   f3 0f 10 04 be          movss  (%rsi,%rdi,4),%xmm0
  4011fb:   f3 0f 58 04 b9          addss  (%rcx,%rdi,4),%xmm0
  401200:   f3 0f 11 04 ba          movss  %xmm0,(%rdx,%rdi,4)
  401205:   8d 78 01                lea    0x1(%rax),%edi
  401208:   41 39 f8                cmp    %edi,%r8d
  40120b:   7e 5e                   jle    40126b <soma_gpu._omp_fn.2+0x10b>
  40120d:   48 63 ff                movslq %edi,%rdi
  401210:   83 c0 02                add    $0x2,%eax
  401213:   f3 0f 10 04 be          movss  (%rsi,%rdi,4),%xmm0
  401218:   f3 0f 58 04 b9          addss  (%rcx,%rdi,4),%xmm0
  40121d:   f3 0f 11 04 ba          movss  %xmm0,(%rdx,%rdi,4)
  401222:   41 39 c0                cmp    %eax,%r8d
  401225:   7e 44                   jle    40126b <soma_gpu._omp_fn.2+0x10b>
  401227:   48 98                   cltq
  401229:   f3 0f 10 04 86          movss  (%rsi,%rax,4),%xmm0
  40122e:   f3 0f 58 04 81          addss  (%rcx,%rax,4),%xmm0
  401233:   f3 0f 11 04 82          movss  %xmm0,(%rdx,%rax,4)
  401238:   c3                      ret
  401239:   0f 1f 80 00 00 00 00    nopl   0x0(%rax)
  401240:   89 c7                   mov    %eax,%edi
  401242:   31 c0                   xor    %eax,%eax
  401244:   eb 0d                   jmp    401253 <soma_gpu._omp_fn.2+0xf3>
  401246:   66 2e 0f 1f 84 00 00    cs nopw 0x0(%rax,%rax,1)
  40124d:   00 00 00
  401250:   4c 89 c0                mov    %r8,%rax
  401253:   f3 0f 10 04 86          movss  (%rsi,%rax,4),%xmm0
  401258:   f3 0f 58 04 81          addss  (%rcx,%rax,4),%xmm0
  40125d:   4c 8d 40 01             lea    0x1(%rax),%r8
  401261:   f3 0f 11 04 82          movss  %xmm0,(%rdx,%rax,4)
  401266:   48 39 f8                cmp    %rdi,%rax
  401269:   75 e5                   jne    401250 <soma_gpu._omp_fn.2+0xf0>
  40126b:   c3                      ret

Disassembly of section .fini:
```

G Código completo de geração do gráfico e das tabelas

O script lê os sete arquivos de saída, confere os parâmetros e os erros e gera os CSVs, as tabelas e o gráfico. Não recompila nem executa o programa de soma.

```
"""Extrai os logs existentes e gera tabelas e gráficos, sem executar t18."""
from pathlib import Path
import csv
import re
from statistics import mean, stdev

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
DATA, FIG, TABLE = (ROOT / name for name in ("dados", "figuras", "tabelas"))
FIG.mkdir(exist_ok=True)
TABLE.mkdir(exist_ok=True)
SERIES = (
    ("CPU1a.out", "CPU 1a", "CPU", 1, "service3", "CPU não registrada"),
    ("CPU1b.out", "CPU 1b", "CPU", 1, "service3", "CPU não registrada"),
    ("CPU4.out", "CPU 4", "CPU", 4, "service3", "CPU não registrada"),
    ("GPUa.out", "V100 a", "GPU", None, "gpunode1", "Tesla V100-SXM2-16GB"),
    ("GPUb.out", "V100 b", "GPU", None, "gpunode1", "Tesla V100-SXM2-16GB"),
    ("resultado_gpu_2133025.out", "H200", "GPU", None, "gpunode4", "NVIDIA H200"),
    ("resultado_gpu_2134904.out", "V100 / 4 visíveis", "GPU", None, "gpunode1",
     "Tesla V100-SXM2-16GB"),
)
records, summaries = [], []
for filename, label, mode, threads, host, model in SERIES:
    text = (DATA / "logs" / filename).read_text(encoding="utf-8")
    assert f"Hostname: {host}.npad.internal" in text
    chunks = re.split(r"----- (Warm-up|Execucao \d+) -----", text)
    times, warmup = [], None
    for i in range(1, len(chunks), 2):
        stage, block = chunks[i:i + 2]
        n = int(re.search(r"N\s*:\s*(\d+)", block)[1])
        tol = float(re.search(r"TOL\s*:\s*([\deE+.-]+)", block)[1])
        declared = re.search(r"MODO\s*:\s*(CPU|GPU)", block)[1]
        errors = int(re.search(r"Erros\s*:\s*(\d+)", block)[1])
        elapsed = float(re.search(r"Tempo\s*:\s*([\d.]+)", block)[1])
        assert n == 10000000 and tol == 1e-9 and declared == mode and errors == 0
        repetition = 0 if stage == "Warm-up" else int(stage.split()[-1])
        records.append((filename, label, mode, threads, host, repetition,
                        n, tol, errors, elapsed))
        if repetition == 0:
            warmup = elapsed
        else:
            times.append(elapsed)
    assert len(times) == 5
    avg, sd = mean(times), stdev(times)
    summaries.append(dict(file=filename, label=label, mode=mode, threads=threads,
                          host=host, model=model, warmup=warmup, times=times,
                          avg=avg, sd=sd, cv=100 * sd / avg))
assert len(records) == 39

with (DATA / "tempos_extraidos.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";", lineterminator="\n")
    writer.writerow(("arquivo", "serie", "modo_declarado", "threads_configuradas",
                     "hostname", "repeticao_0_aquecimento", "N", "tolerancia",
                     "erros", "tempo_segundos"))
    writer.writerows(records)
with (DATA / "resumo.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";", lineterminator="\n")
    writer.writerow(("arquivo", "serie", "media_segundos", "desvio_amostral",
                     "cv_percentual", "aquecimento_segundos"))
    for row in summaries:
```

```
        writer.writerow((row["file"], row["label"], row["avg"], row["sd"],
                         row["cv"], row["warmup"]))


def pt(number, digits=3):
    return f"{number:.{digits}f}".replace(".", ",")


def table(name, columns, heading, caption, rows, label):
    text = [r"\begin{longtable}{" + columns + "}", r"\caption{" + caption +
            r"}\label{" + label + r"}\\", r"\toprule " + heading + r"\\",
            r"\midrule\endfirsthead", r"\toprule " + heading + r"\\",
            r"\midrule\endhead", r"\bottomrule\endfoot"]
    text.extend(" & ".join(row) + r"\\" for row in rows)
    text.append(r"\end{longtable}")
    (TABLE / name).write_text("\n".join(text) + "\n", encoding="utf-8")


table("resumo.tex", "lrrrr",
      r"Série & Média (ms) & $s$ (ms) & CV (\%) & Aquecimento (ms)",
      "Cinco medições por série; aquecimentos separados e excluídos das médias. "
      "Os nomes de GPU identificam o hardware listado no nó, sem confirmar offload.",
      [[row["label"], pt(row["avg"] * 1000, 6), pt(row["sd"] * 1000, 6),
        pt(row["cv"], 2), "---" if row["warmup"] is None else pt(row["warmup"] * 1000, 6)]
       for row in summaries], "tab:resumo")
table("brutos.tex", "lrrr",
      r"Série & Repetição & Tempo (s) & Erros",
      "Todos os 39 registros. Repetição 0 identifica o aquecimento em processo separado.",
      [[row[1], str(row[5]), pt(row[9], 9), str(row[8])] for row in records], "tab:brutos")
table("cpu.tex", "lrrr",
      r"Referência de uma thread & $T_1/T_4$ & $E_4$ (\%) & $T_4$ (ms)",
      "Comparação CPU com o mesmo N. As duas referências permanecem separadas.",
      [[row["label"], pt(row["avg"] / summaries[2]["avg"], 4),
        pt(100 * row["avg"] / summaries[2]["avg"] / 4, 2),
        pt(summaries[2]["avg"] * 1000, 6)] for row in summaries[:2]], "tab:cpu")

plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "pdf.fonttype": 42})
fig, ax = plt.subplots(figsize=(8.5, 4.2), layout="constrained")
for i, row in enumerate(summaries):
    color = "#2366a8" if row["mode"] == "CPU" else "#d4791f"
    ax.barh(i, row["avg"] * 1000, color=color, alpha=.65, height=.58)
    ax.errorbar(row["avg"] * 1000, i, xerr=row["sd"] * 1000,
                fmt="none", ecolor="black", capsize=4)
    ax.scatter([value * 1000 for value in row["times"]], [i] * 5,
               color=color, s=25, zorder=3)
ax.set(yticks=range(7), yticklabels=[row["label"] for row in summaries],
       xlabel="Tempo da chamada medida (ms)", xlim=(0, 15))
ax.invert_yaxis()
ax.grid(axis="x", alpha=.22)
fig.savefig(FIG / "tempos.pdf", bbox_inches="tight")
plt.close(fig)
print("Sete séries, 35 medições e quatro aquecimentos extraídos e conferidos.")
```

Referências

[1] Material da disciplina de Programação Paralela. *Tarefa 18: Comparando soma de vetores entre CPU e GPU em OpenMP*. Enunciado reproduzido no Apêndice A.

[2] UNIVERSITY OF BRISTOL; NPAD/UFRN. *Programming Your GPU with OpenMP*. Tutorial e adaptação NPAD. Versão NPAD consultada: `a8ceec4`. Disponível em: `https://github.com/UoB-HPC/openmp-tutorial` e `https://github.com/NPAD-UFRN/openmp-tutorial`.

[3] PINTO, Matheus Bezeril. *ProgParalela2026.2: Tarefa 18*. Fontes e resultados enviados no commit `e17b7d0`. Disponível em: `https://github.com/39Matheus/ProgParalela2026.2/tree/e17b7d0/Tarefa_18`.

[4] NPAD/UFRN. *Hardware*. Especificações de referência indicadas e transcritas pelo autor em 18 de outubro de 2026. Trecho utilizado reproduzido no Apêndice A. Disponível em: `https://npad.ufrn.br/npad/hardware`.

[5] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface*. Conceitos de target, loop, mapeamento, reduction e consultas ao dispositivo. Disponível em: `https://www.openmp.org/specifications/`.